Universidad de San Carlos de Guatemala

Centro Universitario de Occidente

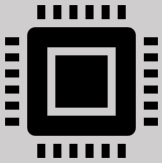
División de Ciencias de la ingeniería

Ingeniería

Manejo e implementación de archivos Sección “A”

Ing. Bryan Misael Monzón Fuentes y Auxiliar Levi Hernández

Segundo Semestre 2026



Estudiante:

Pablo Alejandro Maldonado de León

Carné: 202430233

Modelo documental Proyecto #2



Descripción:

En el siguiente documento se presentan por completo todas las tablas de la decisión de negocio tomada para llevar a cabo el desarrollo del sistema. Este documento es un respaldo del trabajo generado ya que fue en base un modelo planteado de gestión de documentos. Ya que al final esta base de datos utilizada no contaba con relaciones directas por lo tanto fue algo difícil visualizar esas relaciones que se tenían. Sin embargo tras varios días de planificación se pudo desarrollar un sistema robusto con ese tipo de implementaciones para generar un NAS.

Disclaimer: Todos los campos se encuentran escritos y mapeados en inglés

“Bienvenido lector”

Para este proyecto se utilizó una base de datos NoSQL cuya estructura quedó a completa elección del estudiante por lo tanto en este manual se le dio a conocer el modelo documental de cada estructura de documentos en MongoDB. Esto con la finalidad de presentar el proyecto y tener el respaldo de las decisiones de negocio tomadas para la correcta gestión de la base de datos y validaciones de integridad.

Consideraciones:

- **MongoDB:** Por defecto MongoDB no permite transacciones por lo que hay que activar una configuración que permita que mongo pueda generar transacciones por lo tanto eso fue utilizado para generar transacciones desde mongo
- **Soporte para guardar archivos:** Debido a que el proyecto consiste en la implementación de un NAS. Esto de subir archivos es un problema. Porque a pesar de ser puros archivos de texto los requests no se podían tratar como si fueran strings simples. Porque podían subir un archivo muy grande y esto mismo podría colapsar la aplicación cliente. Por lo tanto se trataron los documentos para soportar el almacén de archivos con GridF para poder ir cortando en pedazos los archivos y poderlos almacenar en la base de datos.

Modelo de datos documental

A continuación se le dará a conocer la información sobre cómo se implementaron los documentos. Cada documento tiene la estructura en tablas. Este es solo un ejemplo del stack utilizado para poder llevar a cabo el proyecto. Dando una previsualización de lo que se utilizó.

Dato	Detalle
Base de datos	MongoDB
Capa de persistencia	Mongoose con decoradores @Schema y @Prop (NestJS)

Convenciones

- Todas las entidades extienden AuditableEntity y usan timestamps: true, por lo que Mongoose mantiene createdAt y updatedAt. Estos campos no se repiten en cada tabla.
- Todo documento tiene _id de tipo ObjectId, generado por MongoDB.
- La integridad la garantiza el servidor.

Mapa de colecciones

Colección	Propósito
permissions	Catálogo de capacidades globales del sistema
roles	Roles con su lista embebida de códigos de permiso (RBAC)
users	Identidad, rol, estado y cuota de almacenamiento
system_settings	Reglas generales del sistema en formato clave-valor
folders	Árbol de carpetas por propietario
files	Metadatos de archivos txt y md
shares	Permisos por recurso y destinatario (ACL)
refresh_tokens	Control de sesiones y ciclos de vida de JWT
comments	Comentarios e hilos sobre archivos. Existía en el diseño conceptual
activity_logs	Historial inmutable de actividad. Y control de actividad de archivos
fs.files / fs.chunks	Contenido binario; lo gestiona GridFS, no se modela a mano

Relaciones

A continuación se presentan las relaciones que se establecen en los documentos para poder garantizar la integridad de la información.

Origen	Campo	Destino	Notas
users	roleId	roles	Obligatoria. El rol cambia de forma independiente al usuario
folders	ownerId	users	Propietario de la carpeta
folders	parentId	folders	null = carpeta de primer nivel
folders	ancestors[]	folders	Cadena de ancestros, de la raíz hacia el padre
folders	deletedBy	users	Quién envió a papelera
files	ownerId	users	Propietario del archivo
files	folderId	folders	null = raíz virtual del usuario
files	deletedBy	users	Quién envió a papelera
files	gridFsId	fs.files	Sin ref declarado; apunta al contenido en GridFS
shares	ownerId	users	Dueño del recurso, copiado al compartir
shares	granteeId	users	null cuando el destinatario es acceso general
shares	resourceId	files o folders	Polimórfica: depende de resourceType
system_settings	updatedBy	users	Sin ref declarado
roles	permissions[]	permissions.code	Referencia por valor, no por ObjectId

Datos embebidos

Documento padre	Embebido	Motivo
users	storage (quotaBytes, usedBytes)	Uno a uno, sin vida propia. Dos contadores que se actualizan de forma atómica sobre un solo documento, lo que evita carreras al verificar la cuota.
roles	permissions (arreglo de códigos)	Lista pequeña y acotada que siempre se lee junto con el rol
folders	ancestors (arreglo de ObjectId)	Permite consultar el subárbol completo con una sola consulta, sin recursión

DOCUMENTOS IMPLEMENTADOS

Permisos:

Catálogo de capacidades globales (RBAC). Sirve para validar qué códigos puede llevar un rol.

Campo	Tipo	Obligatorio	Notas
code	string	Sí	Identificador estable del permiso
module	string	Sí	Agrupa permisos para mostrarlos
name	string	Sí	Nombre legible
description	string	Sí	

Roles

Cada rol lleva embebida la lista de códigos de permiso que concede. Un rol nuevo con un subconjunto de permisos no requiere cambios de esquema.

Campo	Tipo	Obligatorio	Notas
code	string	Sí	Ejemplos: ADMIN, USER
name	string	Sí	
description	string	No	
permissions	string[]	No	Códigos del catálogo permissions. Embebido
isSystem	boolean	No	Protege los roles base de ser borrados
isActive	boolean	No	Permite desactivar un rol sin borrarlo
deactivatedAt	Date	No	Fecha de desactivación

Usuarios:

Identidad, rol, estado y cuota de almacenamiento. La identidad real es el id; username y email son editables.

Campo	Tipo	Obligatorio.	Notas
username	string	Sí	
email	string	Sí	
passwordHash	string	Sí	Nunca texto plano. Se excluye de las lecturas por defecto
firstName	string	Sí	
lastName	string	Sí	
roleId	ObjectId	Sí	Referencia al rol
isActive	boolean	No	
deactivatedAt	Date	No	Desactivar no borra nada
lastLoginAt	Date	No	Último inicio de sesión
storage	objeto embebido	Sí	Ver abajo

Documento embebido (almacenamiento):

Campo	Tipo	Obligatorio	Notas
storage.quotaBytes	number	Sí	Cuota real del usuario, copiada del valor por defecto al crearlo
storage.usedBytes	number	Sí	Contador atómico. Incluye lo que está en papelera

Configuraciones del sistema

Un documento por regla del sistema. Permite agregar reglas sin cambiar esquema y registrar quién modificó cada una.

Campo	Tipo	Obligatorio	Notas
key	string	Sí	storage.defaultQuotaBytes. No declara unicidad en la entidad
label	string	Sí	Nombre legible para pantallas de administración
category	string	Sí	Agrupa reglas por categoría
value	Mixed (string o number)	Sí	Se valida en el servicio según la clave
description	string	Sí	
updatedBy	ObjectId	No	Usuario que modificó la regla. Sin ref declarado

Regla de cuota: Guarda el valor por defecto para usuarios nuevos. Guarda la cuota real de cada usuario. Cambiar el valor por defecto solo afecta a usuarios futuros. Subir la cuota de un usuario siempre se permite; bajarla solo si el nuevo valor es mayor o igual a usedBytes.

Folders:

Árbol de carpetas por propietario, con referencia al padre más un arreglo de ancestros. La papelera es una marca lógica en el propio documento.

Campo	Tipo	Obligatorio	Notas
ownerId	ObjectId	Sí	
parentId	ObjectId o null	No	null = carpeta de primer nivel
name	string	Sí	Debe ser único entre hermanos activos
ancestors	ObjectId[]	No	De la raíz hacia el padre
isDeleted	boolean	No	Marca de papelera
deletedAt	Date	No	
deletedBy	ObjectId	No	
trashBatchId	ObjectId	No	Agrupa todo lo eliminado en una operación
isTrashRoot	boolean	No	Verdadero sólo en lo que el usuario elimina directamente

Archivos:

Metadatos de archivos. Los metadatos son la fuente de verdad; GridFS solo guarda el binario, por lo que renombrar o mover toca únicamente este documento.

Campo	Tipo	Obligatorio	Notas
ownerId	ObjectId	Sí	
folderId	ObjectId o null	No	null = raíz virtual del usuario
name	string	Sí	Con extensión
extension	string	Sí	Es la clasificación del archivo
mimeType	string	Sí	
sizeBytes	number	Sí	Cambia en cada edición
gridFsId	ObjectId	Sí	Apunta al contenido en fs.files
isDeleted	boolean	No	Marca de papelera
deletedAt	Date	No	
deletedBy	ObjectId	No	
trashBatchId	ObjectId	No	Mismo valor para todo el lote
isTrashRoot	boolean	No	

Compartidos (ACL):

Una fila por recurso y destinatario. Es el estado actual de los permisos, el historial vive en los logs de actividad.

Campo	Tipo	Obligatorio	Notas
resourceType	string	Sí	Define a qué colección apunta resourceId
resourceId	ObjectId	Sí	Sin referencia fuerte
ownerId	ObjectId	Sí	Dueño del recurso, copiado al compartir
granteeType	string	Sí	USER o ANY_USER (cualquier usuario autenticado)
granteeId	ObjectId o null	No	null cuando es ANY_USER
access	string	Sí	VIEWER, COMMENTER, EDITOR

5. Índices declarados en las entidades

Lista de los índices únicos. Los índices compuestos o parciales, si existen, se declaran fuera de este archivo y no se pueden ver aquí.

Colección	Índice	Para qué
permissions	code	Unicidad del código
roles	code	Unicidad del código
users	username	Login y unicidad
users	email	Login y unicidad
users	roleId	Consultar usuarios por rol
folders	ownerId	Carpetas de un usuario
folders	ancestors	Subárbol de una carpeta
folders	isDeleted; trashBatchId; isTrashRoot	Papelera
files	ownerId; folderId	Archivos de un usuario y contenido de una carpeta
files	extension	Reportes por tipo
files	isDeleted; trashBatchId; isTrashRoot	Papelera
shares	resourceType; resourceId	Permisos de un recurso
shares	ownerId	Recursos compartidos por un usuario
shares	granteeId	Recursos recibidos por un usuario

Reglas de modelado

Gestión de permisos en dos capas

- **RBAC**: Es la lógica sobre la que está montado toda la aplicación cliente. Responde si el usuario puede ejecutar una operación global del sistema. Solo quien tenga el permiso de gestión de usuarios cambia el rol de otro usuario.
- **ACL**: responde si el usuario puede hacer algo sobre un archivo o carpeta concreto.
- Una petición sobre un recurso pasa por ambas capas, que no se pisan. Es decir que ambas implementaciones conviven en armonía y generan robustez al sistema.

6.2 Reglas de la ACL

- Sin ninguna fila en shares, el recurso es restringido para todos salvo el propietario.
- Un destinatario tiene un solo nivel por recurso: cambiarlo y actualizar su fila.
- El nivel más alto gana: LECTOR < COMENTADOR < EDITOR. Aplica entre fila directa, fila heredada de carpeta y acceso general.
- Los permisos manuales y el acceso general ANY_USER conviven como filas independientes. Desactivar el general solo borra su fila.
- Compartir una carpeta da acceso a su subárbol: se consulta shares por el recurso o por cualquiera de sus ancestros.
- Solo el propietario comparte y administra permisos. Retirar un permiso borra la fila.
- Si el recurso está en papelera, los permisos se ignoran.

Árbol de carpetas

- La raíz de cada usuario ("Mi unidad") es virtual: es el valor null en parentId y en folderId. No es un documento ni aparece en los ancestros.
- Al mover una carpeta se rechaza el movimiento si el destino es ella misma o la contiene en sus ancestros.
- Mover una carpeta reescribe ancestros de todo su subárbol, en transacción.
- Mover un archivo o carpeta cambia sus permisos heredados, porque cambia la cadena de ancestros. Los permisos directos se conservan.

Papelera

- Marca lógica en el propio documento: isDeleted, deletedAt, deletedBy, trashBatchId e isTrashRoot.
- Eliminar una carpeta marca ella y todo su subárbol con el mismo trashBatchId. La vista de papelera lista solo los isTrashRoot.
- Restaurar reactiva el lote completo. Si el padre original ya no existe o está en papelera, se restaura en la raíz del usuario; si hay conflicto de nombre, se renombra automáticamente.
- La eliminación definitiva borra documentos y contenido en GridFS, descuenta sizeBytes de usedBytes y elimina los shares y comments asociados.

LÓGICA QUE DEFINE EL NEGOCIO

Cuota:

- Lo que está en papelera sigue contando en usedBytes. Sólo la eliminación definitiva descuenta el espacio.
- Si un tercero con permiso EDITOR edita un archivo, la diferencia de tamaño se cobra al propietario.

Transacciones

Requieren transacción: subir archivo, editar contenido, mover carpeta, enviar una carpeta a papelera, restaurar, eliminación definitiva y compartir con varias personas a la vez.

Nombres de entidad en el código backend

Colección	Clase de entidad	Tipo de documento	Esquema
permissions	PermissionEntity	PermissionDocument	PermissionSchema
roles	RoleEntity	RoleDocument	RoleSchema
users	UserEntity	UserDocument	UserSchema
users.storage	StorageEntity	Embebido	StorageSchema
system_settings	SystemSettingEntity	SystemSettingDocument	SystemSettingSchema
folders	FolderEntity	FolderDocument	FolderSchema
files	FileEntity	FileDocument	FileSchema
shares	ShareEntity	ShareDocument	ShareSchema